


# UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA

# CLASIFICACIÓN DE TRES REQUERIMIENTOS DEL PROYECTO ÓRBIKA MEDIANTE EL DESARROLLO DE SOFTWARE DIRIGIDO POR MODELOS (MDD)

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

***Renzo Fernando Loyola Vilca*** ***(2021072615)***

***Brant Antony Chata Choque*** ***(2020067577)***

**Tacna – Perú**

***2026***

## Contenido


1. INTRODUCCIÓN 3
2. DESCRIPCIÓN DEL PROYECTO ÓRBIKA 3
3. FUNDAMENTOS DEL DESARROLLO DE SOFTWARE DIRIGIDO POR MODELOS 4
4. CRITERIOS PARA SELECCIONAR LOS REQUERIMIENTOS 4
5. DESARROLLO Y CLASIFICACIÓN DE LOS TRES REQUERIMIENTOS 4
5.1 Requerimiento 1: Publicar un producto perecible (CU-07) 4
Descripción del requerimiento 4
Modelo CIM: negocio y reglas 5
Modelo PIM: diseño lógico 5
Modelo PSM: adaptación tecnológica 5
Diagrama textual del modelo 5
5.2 Requerimiento 2: Comprar un producto (CU-05) 6
Descripción del requerimiento 6
Modelo CIM: negocio y reglas 6
Modelo PIM: diseño lógico 6
Modelo PSM: adaptación tecnológica 6
Diagrama textual del modelo 7
5.3 Requerimiento 3: Cancelar un pedido (CU-17) 7
Descripción del requerimiento 7
Modelo CIM: negocio y reglas 7
Modelo PIM: diseño lógico 8
Modelo PSM: adaptación tecnológica 8
Diagrama textual del modelo 9
6. COMPARACIÓN DE LOS MODELOS CIM, PIM Y PSM 10
7. CONCLUSIONES 11
8. RECOMENDACIONES 11
9. BIBLIOGRAFÍA 11

## 1. INTRODUCCIÓN

El presente trabajo aplica el enfoque de Desarrollo de Software Dirigido por Modelos (Model-Driven Development, MDD) al proyecto ÓrbiKa, una plataforma web orientada a la comercialización y revalorización de productos perecibles en la provincia de Tacna. La plataforma busca conectar a vendedores que ofrecen alimentos próximos a vencer con compradores interesados en adquirirlos a precios accesibles, promoviendo el aprovechamiento de alimentos y el consumo sostenible.

El objetivo es clasificar y representar tres requerimientos del sistema en diferentes niveles de abstracción: el modelo independiente de la computación o del negocio (CIM), el modelo independiente de la plataforma (PIM) y el modelo específico de plataforma (PSM). Para ello, se describen el contexto y las reglas del negocio, se identifican las clases y relaciones principales y se explica cómo se podrían implementar con las tecnologías utilizadas por la versión actual del repositorio.

El análisis toma como base los documentos de requerimientos y arquitectura incluidos en el repositorio, así como el README técnico de orbika-app. Se distingue entre el alcance descrito en los documentos académicos y el estado de implementación declarado por el propio repositorio.

## 2. DESCRIPCIÓN DEL PROYECTO ÓRBIKA

ÓrbiKa es un mercado web de alimentos perecibles para Tacna. Sus actores principales son Comprador, Vendedor y Administrador. Entre las funciones descritas por los documentos del proyecto están el catálogo de productos, publicación y edición de ofertas, compra, seguimiento y cancelación de pedidos, calificación de transacciones, administración y reportes de impacto.

La implementación actual descrita en orbika-app utiliza React 18, Vite y Tailwind CSS en el frontend; una API REST construida con Node.js y Express en el backend; y Supabase para autenticación y PostgreSQL

| **Elemento** | **Descripción** |
|---|---|
| Problema | Parte de los alimentos perecibles puede desperdiciarse cuando se aproxima su fecha de vencimiento. |
| Propuesta | Facilitar la publicación y compra de productos perecibles a precios accesibles. |
| Actores principales | Comprador, Vendedor y Administrador. |
| Tecnologías actuales declaradas | React 18, Vite, Tailwind CSS, Node.js, Express, Supabase Auth, PostgreSQL. |
| Alcance de este informe | Modelar tres requerimientos: publicar producto, comprar producto y cancelar pedido. |

## 3. FUNDAMENTOS DEL DESARROLLO DE SOFTWARE DIRIGIDO POR MODELOS

El Desarrollo de Software Dirigido por Modelos (MDD) utiliza modelos como elementos centrales del proceso de desarrollo. En lugar de comenzar directamente con el código, primero se representa el negocio y sus reglas; después se construye un diseño lógico del sistema y finalmente se adapta ese diseño a una plataforma tecnológica concreta.

| Nivel | Nombre | Aplicación a ÓrbiKa |
|---|---|---|
| CIM | Computation-Independent Model / modelo independiente de la computación | Describe las necesidades del negocio, los actores, los procesos y las reglas sin depender de una tecnología. |
| PIM | Platform-Independent Model / modelo independiente de la plataforma | Representa la estructura lógica del software mediante clases, atributos, métodos y relaciones, sin vincularlo a un lenguaje específico. |
| PSM | Platform-Specific Model / modelo específico de plataforma | Adapta el diseño a tecnologías concretas, por ejemplo, React, Node.js/Express, Supabase y PostgreSQL. |

La transformación CIM → PIM → PSM permite mantener trazabilidad entre la necesidad del negocio, el diseño del software y su implementación. Los modelos no sustituyen las pruebas: deben validarse frente a los requerimientos y las reglas del sistema.

## 4. CRITERIOS PARA SELECCIONAR LOS REQUERIMIENTOS

| Código de trabajo | Requerimiento seleccionado | Relación con el proyecto |
|---|---|---|
| R1 / CU-07 | Publicar un producto perecible | El vendedor registra una oferta con precio, stock, fecha de vencimiento y modalidad de entrega. |
| R2 / CU-05 | Comprar un producto | El comprador confirma una compra; el sistema crea el pedido y registra el descuento de stock. |
| R3 / CU-17 | Cancelar un pedido | Se cancela un pedido permitido por las reglas y se restituyen las unidades sin duplicar la operación. |

## 5. DESARROLLO Y CLASIFICACIÓN DE LOS TRES REQUERIMIENTOS

### 5.1 Requerimiento 1: Publicar un producto perecible (CU-07)

#### Descripción del requerimiento

El sistema debe permitir que un vendedor habilitado registre un producto perecible para ofrecerlo en el catálogo. La información relevante comprende nombre, descripción, categoría, fotografía, precio original, precio actual, stock disponible, fecha de vencimiento, peso por unidad, zona y modalidad de entrega. El sistema calcula el descuento y determina si el producto está disponible según las reglas definidas.

### Modelo CIM: negocio y reglas

Actor principal: Vendedor. Objetivo: publicar una oferta real y comprensible para los compradores. Entidades del negocio: Vendedor, Producto y Categoría. Reglas relevantes: el precio original y el precio actual deben ser válidos; el stock debe permitir la venta; la fecha de vencimiento debe ser posterior al día actual de Lima para que la oferta se considere disponible; el producto debe asociarse a una categoría habilitada. La publicación no debe considerarse exitosa si el servidor rechaza la operación.

### Modelo PIM: diseño lógico

| Clase | Atributos principales | Responsabilidades / métodos |
|---|---|---|
| Vendedor | idUsuario, nombreNegocio, ubicaciónNegocio | publicarProducto(), editarProducto() |
| Producto | idProducto, nombre, descripción, precioOriginal, precioActual, stockDisponible, fechaVencimiento, pesoUnidadKg, zona, estadoPublicación | calcularDescuento(), comprobarDisponibilidad(), actualizarDatos() |
| Categoría | idCategoría, nombre, estado | validarCategoríaActiva() |

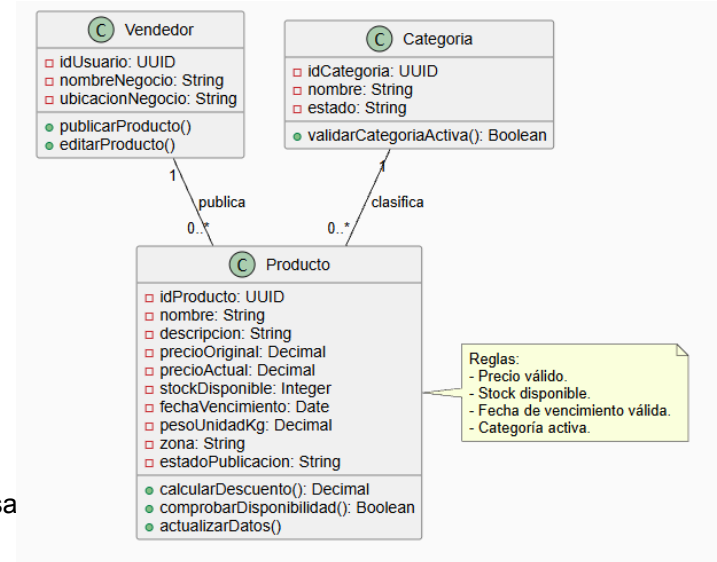
Relaciones principales: un Vendedor puede publicar varios Productos; cada Producto pertenece a una Categoría. La clase Producto concentra los datos de la oferta y las reglas que permiten determinar su disponibilidad.

### Modelo PSM: adaptación tecnológica

En la implementación actual, la interfaz React presenta el formulario de publicación y valida los datos antes de enviarlos a la API. El backend Node.js/Express recibe la solicitud mediante una ruta y su controlador de productos, valida la operación y utiliza los servicios de Supabase/PostgreSQL para persistir los datos. Las imágenes se envían a la API y se almacenan en Supabase Storage; el sistema guarda la URL devuelta. Los nombres concretos de tablas y rutas deben verificarse en los archivos fuente y migraciones de la versión entregada.

Flujo resumido: Vendedor → formulario de publicación → API de productos → validación de reglas → PostgreSQL y almacenamiento de imagen → respuesta de éxito o error.

### Diagrama textual del modelo



## 5.2 Requerimiento 2: Comprar un producto (CU-05)

### Descripción del requerimiento

El sistema debe permitir que un comprador seleccione un producto disponible, indique una cantidad válida y confirme la compra. La operación registra un pedido y su detalle, conserva los valores históricos de la transacción y descuenta el stock. La compra debe impedir cantidades inválidas o ventas que superen el stock disponible.

### Modelo CIM: negocio y reglas

Actor principal: Comprador. Entidades del negocio: Comprador, Producto, Pedido y DetallePedido. Reglas: el producto debe estar publicado y disponible; debe existir stock suficiente; la cantidad debe ser positiva; el pedido debe guardar el precio aplicado y el valor necesario para mantener el historial de la compra. El README indica que la creación del pedido, el detalle, el descuento de stock y el aviso se ejecutan en una misma transacción de base de datos. También señala que la operación usa una clave UUID de confirmación para gestionar reintentos.

### Modelo PIM: diseño lógico

| Clase | Atributos principales | Responsabilidades / métodos |
|---|---|---|
| Comprador / Usuario | idUsuario, nombre, correo, rol, estado | autenticar(), consultarCatálogo() |
| Producto | idProducto, precioActual, precioOriginal, stockDisponible, fechaVencimiento, estadoPublicación | comprobarDisponibilidad(), descontarStock() |
| Pedido | idPedido, fechaCreación, estado, modalidadEntrega, referenciaEntrega, montoTotal | registrarCompra(), avanzarEstado() |
| DetallePedido | pedido, producto, cantidad, precioCompra, precioOriginalCompra, pesoUnidadCompraKg, subtotal | calcularSubtotal() |
| Notificación | idNotificación, destinatario, mensaje, fechaEnvío, leída | crearAviso(), marcarLeída() |

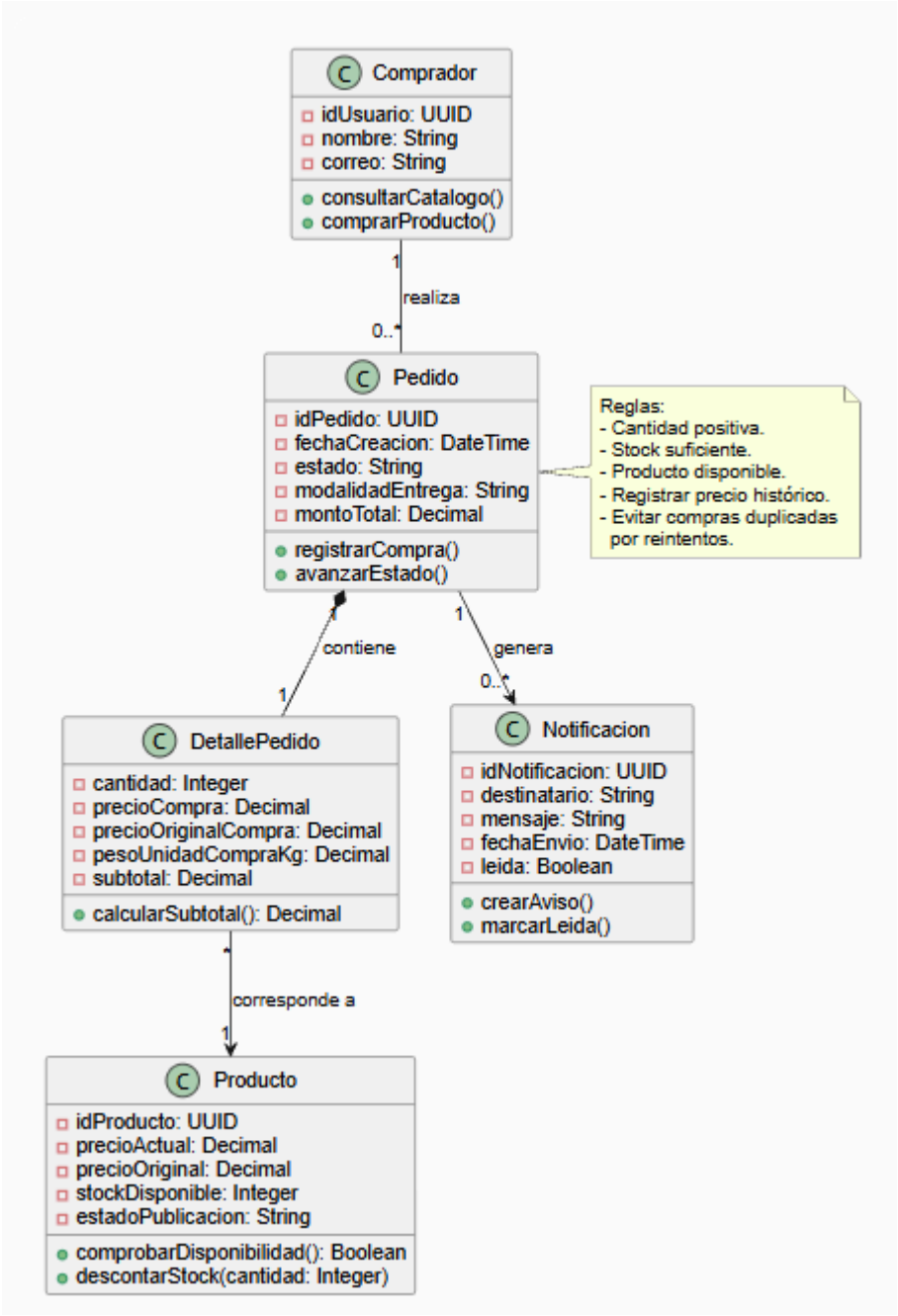
Relaciones: un Comprador puede tener varios Pedidos; en el diseño del SRS vigente descrito por el repositorio, cada Pedido tiene un único DetallePedido; el detalle se refiere a un Producto; y la operación puede generar una Notificación. Se conserva el precio histórico de compra para no alterar pedidos antiguos cuando cambie el precio de una publicación.

### Modelo PSM: adaptación tecnológica

La interfaz React solicita la compra a la API Node.js/Express. El backend valida los datos y ejecuta la operación de compra contra Supabase/PostgreSQL. Las migraciones del repositorio describen funciones SQL para crear el pedido, su detalle y descontar stock en una transacción. La API solo debe confirmar la compra después de recibir una respuesta satisfactoria. El sistema no incorpora una pasarela de pago integrada: el importe del pedido no equivale a un pago electrónico acreditado por ÓrbiKa.

Flujo resumido: Comprador → selección y cantidad → API de pedidos → validación de disponibilidad y stock → transacción en PostgreSQL → pedido, detalle, descuento de stock y notificación → confirmación.

### Diagrama textual del modelo




## 5.3 Requerimiento 3: Cancelar un pedido (CU-17)

### Descripción del requerimiento

El sistema debe permitir cancelar un pedido cuando su estado lo permita, registrar la cancelación y restituir el stock correspondiente. La operación debe ser idempotente en el sentido de que una cancelación repetida no devuelva las mismas unidades más de una vez.

### Modelo CIM: negocio y reglas

Actor principal: Comprador, de acuerdo con el flujo de pedidos descrito en el proyecto. Entidades: Comprador, Pedido, DetallePedido y Producto. Reglas: solo se permite cancelar desde los estados 'creado'

o 'preparando'; el sistema registra la fecha y, cuando corresponde, el motivo de cancelación; el stock se restituye una sola vez; no se debe permitir cancelar un pedido ya completado, entregado o cancelado.

### Modelo PIM: diseño lógico

| Clase | Atributos principales | Responsabilidades / métodos |
|---|---|---|
| Pedido | idPedido, comprador, estado, fechaCancelación, motivoCancelación | cancelar(), validarTransiciónEstado() |
| DetallePedido | pedido, producto, cantidad | obtenerCantidadAReponer() |
| Producto | idProducto, stockDisponible | restituirStock(cantidad) |
| Notificación | destinatario, mensaje, fechaEnvío, leída | crearAvisoDeCancelación() |

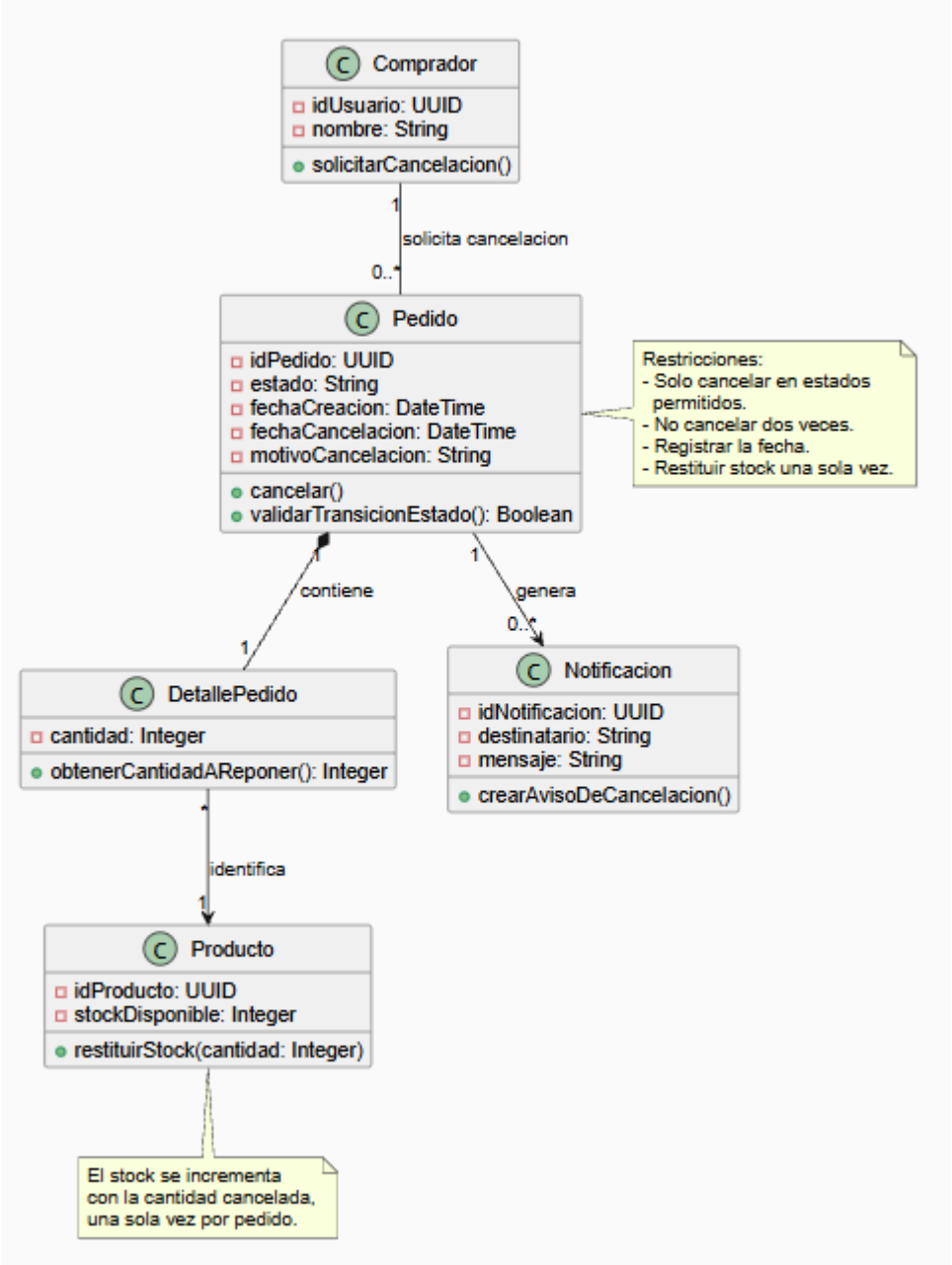
Relaciones: el Pedido contiene la información de la compra; su DetallePedido identifica el producto y la cantidad que debe restituirse; Producto mantiene el stock disponible. El estado del pedido determina si la operación está permitida.

### Modelo PSM: adaptación tecnológica

La interfaz solicita la cancelación a la API de pedidos. El backend comprueba el estado actual y ejecuta la actualización del pedido y la restitución de stock mediante la lógica transaccional definida por el sistema. Debe evitarse que una segunda solicitud de cancelación incremente nuevamente el stock. La respuesta debe comunicar si la cancelación fue aceptada o rechazada y el motivo general del rechazo.

Flujo resumido: Comprador → solicitar cancelación → API de pedidos → comprobar estado → actualizar pedido y restituir stock una sola vez → guardar cambios → respuesta.

### Diagrama textual del modelo

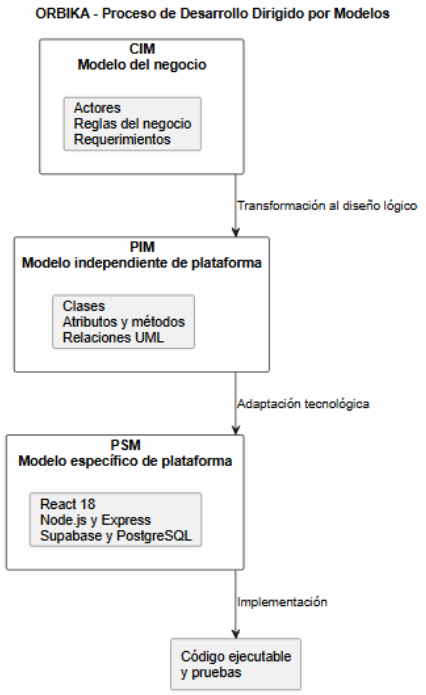



La operación de cancelación modifica el estado de Pedido y la cantidad disponible de Producto. Se recomienda representar en UML una nota de restricción: {solo cancelar si estado = creado o preparando; restituir stock una única vez}.

## 6. COMPARACIÓN DE LOS MODELOS CIM, PIM Y PSM

| Requerimiento | CIM: negocio | PIM: diseño lógico | PSM: tecnología |
|---|---|---|---|
| Publicar producto | Vendedor publica una oferta con información completa y válida. | Vendedor, Producto y Categoría; asociación vendedor-producto y producto-categoría. | Formulario React; API Express; persistencia en PostgreSQL/Supabase Storage. |
| Comprar producto | Comprador adquiere cantidad disponible y el sistema registra la transacción. | Comprador/Usuario, Producto, Pedido, DetallePedido y Notificación. | Frontend React; API de pedidos; transacción SQL en PostgreSQL mediante Supabase. |
| Cancelar pedido | Se permite cancelar solo en estados admitidos y se devuelve el stock una sola vez. | Pedido, DetallePedido y Producto; operación de cambio de estado y reposición. | Solicitud a API Express; validación de estado; actualización transaccional de datos. |

La clasificación evidencia la progresión desde las necesidades del negocio hasta decisiones de implementación. El CIM ayuda a aclarar las reglas; el PIM permite razonar sobre responsabilidades y relaciones sin depender del lenguaje; y el PSM vincula el diseño con las tecnologías reales del proyecto.



## 7. CONCLUSIONES

- El enfoque MDD permite organizar el análisis de ÓrbiKa en niveles de abstracción, relacionando las necesidades del negocio con el diseño lógico y la implementación tecnológica.
- Los tres requerimientos elegidos representan un flujo conectado: publicación de la oferta, compra y eventual cancelación. Sus modelos comparten las entidades Producto y Pedido y las reglas de disponibilidad y stock.
- La definición de reglas explícitas, especialmente para el vencimiento, el stock y los estados del pedido, reduce ambigüedades y ayuda a diseñar pruebas verificables.
- El repositorio declara una implementación parcial de los casos de uso. Por ello, el modelado de un requerimiento no debe confundirse con la afirmación de que toda la funcionalidad está completa o validada en producción.

## 8. RECOMENDACIONES

- Mantener trazabilidad entre cada requerimiento del SRS, su caso de uso, las clases del PIM, las tablas y operaciones del PSM y sus pruebas correspondientes.
- Validar los diagramas UML con el equipo y contrastar los nombres de entidades, atributos, rutas y funciones con el código vigente antes de la entrega final.
- Documentar las reglas de negocio como restricciones verificables: fecha de vencimiento, stock disponible, estados válidos de pedido y prevención de reposición duplicada.
- Probar los flujos de compra y cancelación con casos normales, datos inválidos, reintentos y solicitudes simultáneas en un entorno de pruebas.
- Actualizar los documentos académicos cuando cambie la arquitectura. El README actual indica que el código usa Supabase y no Sequelize/Cloudinary/SMTP propio, por lo que la documentación debe evitar contradicciones entre arquitectura planificada e implementación real.

## 9. BIBLIOGRAFÍA


Grupo G01. (2026). *G01_ORBIKA_FD03_SRS_CHATA_LOYOLA: Documento de Especificación de Requerimientos de Software, versión 3.0* [Documento interno del proyecto]. Universidad Privada de Tacna.

Grupo G01. (2026). *G01_ORBIKA_FD04_SAD_CHATA_LOYOLA: Documento de Arquitectura de Software* [Documento interno del proyecto]. Universidad Privada de Tacna.

Grupo G01. (2026). *ÓrbiKa: README del repositorio orbika-app* [Repositorio de código fuente]. GitHub. https://github.com/renzoloyola/G01_CSW_ORBIKA

Universidad Privada de Tacna. (2026). *Desarrollo de Software Dirigido por Modelos (MDD)* [Material de clase del curso Construcción de Software I].

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759